

Lecture Flow

- 1) Pre-requisites
- 2) Problem definitions and applications
- 3) Types of linked list
- 4) Different approaches
- 5) Checkpoint
- 6) Variants
- 7) Pair Programming
- 8) Recognizing in questions
- 9) Things to pay attention (common pitfalls)
- 10) Practice questions
- 11) Resources
- 12) Quote of the day

Pre-requisites

- Class
- Linear Data Structure

Definitions

Linked List is a linear data structure that stores value and grows dynamically



Node

- stores the **value** and the **reference to the next node**.
- is a collection of two or more sub-elements or parts.
- Is the simplest linked list example

```
class Node:  
    def __init__(self, value):  
        self.value = value  
        self.next = None # Type: Node
```

Singly Linked List is when nodes have only next's node reference.



Why do we need
linked list when we
have **arrays**?



Why Linked List when you have Arrays?

- Arrays by default **don't grow** dynamically
- **Inserting** in the **middle** of an array is **costly**
- **Removing** elements from the **middle** of array is **costly**

Arrays vs Linked List

Array	Linked List
Fixed size	Dynamic size
Insertions and Deletions are inefficient	Insertions and Deletions are efficient
Random access	No random access
Possible waste of memory	No waste of memory
Sequential access is faster	Sequential access is slow

Pros and Cons?

Advantages of using linked list

- Dynamic data structure
- No memory wastage
- Insertion and Deletion Operations

Disadvantages

- Traversal
- Reverse Traversing
- Random Access

Implement Linked List with the given constraints([Link](#)):

- **addAtTail(val)**
 - $O(n)$
 - Append elements
- **print()**
 - $O(n)$
 - Print all elements in their order
- **main()**
 - instantiate, add [1,2,3,4] and print

Dummy Node

- is **a node** that points to the head of a linked list which will be **discarded at the end**
- When to use a Dummy Node?
 - if you are potentially **modifying the head** of linked list, use dummy node

Useful Methods

Traversing a Linked List

- **Start with the head** of the list. Access the content of the head node if it is not null
- go to the **next node(if exists)** and access the node information
- **Continue until no more nodes** (that is, you have reached the last node)

Implementation

```
def traverse(self, head) -> None:  
    currentNode = head  
    while currentNode:  
        print(currentNode.val)  
        currentNode = currentNode.next
```

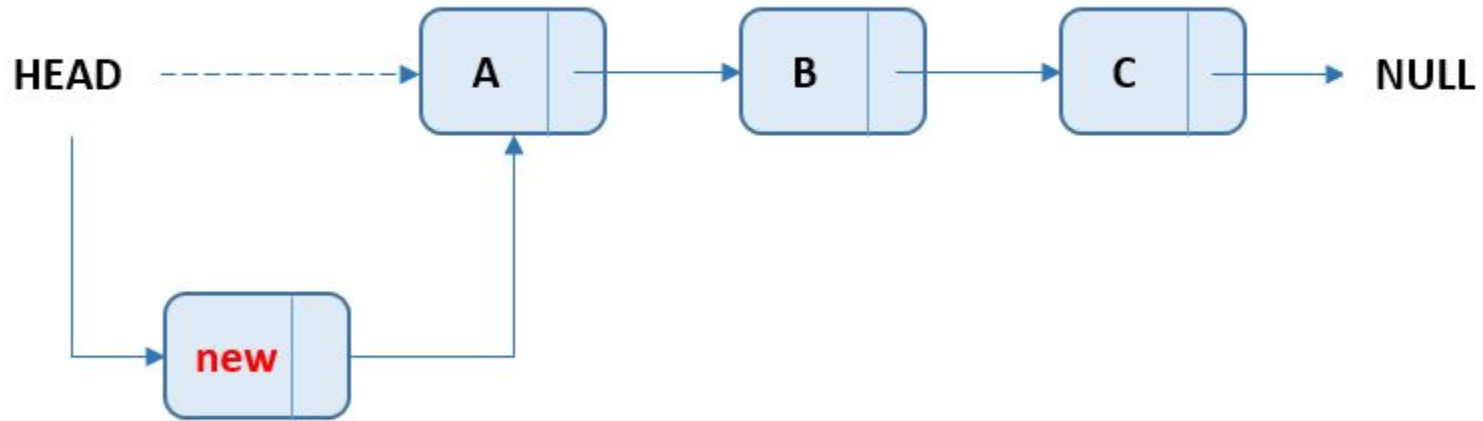
Pair Programming

[Design Linked List](#) (implement **get**)

Inserting a node in linked list

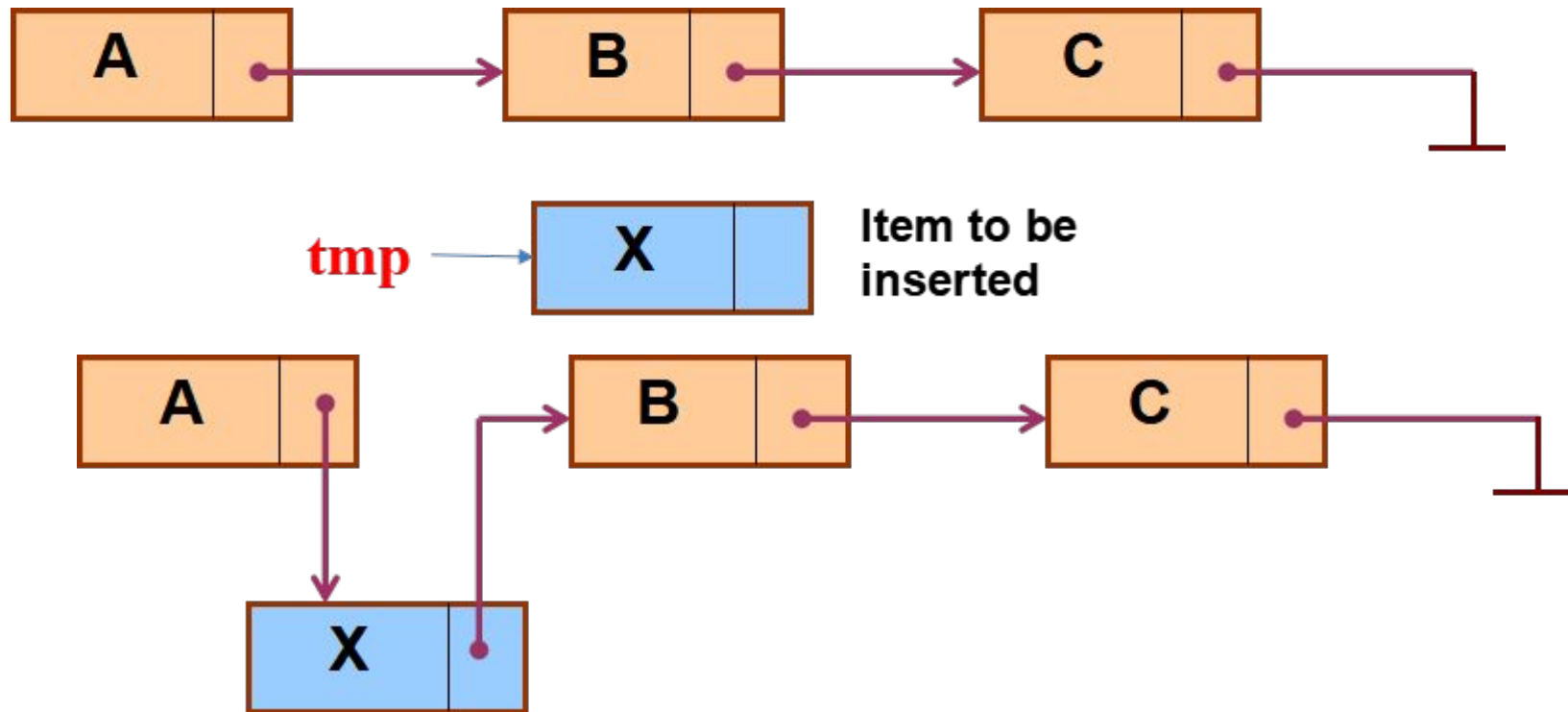
Insert at the beginning

- If list is empty, **make new node the head** of the list
- Otherwise, **connect new node to the current head**
- **make new node the head** of the list.



Insert at any position

- find **the insert position** and **the previous node**
- And then **make the next of new node** as the next of previous node
- finally, **make the next of the previous node** the new node



Can we avoid using two approaches when we are inserting(beginning & any place)? How?

Yes, we can use Dummy Node.

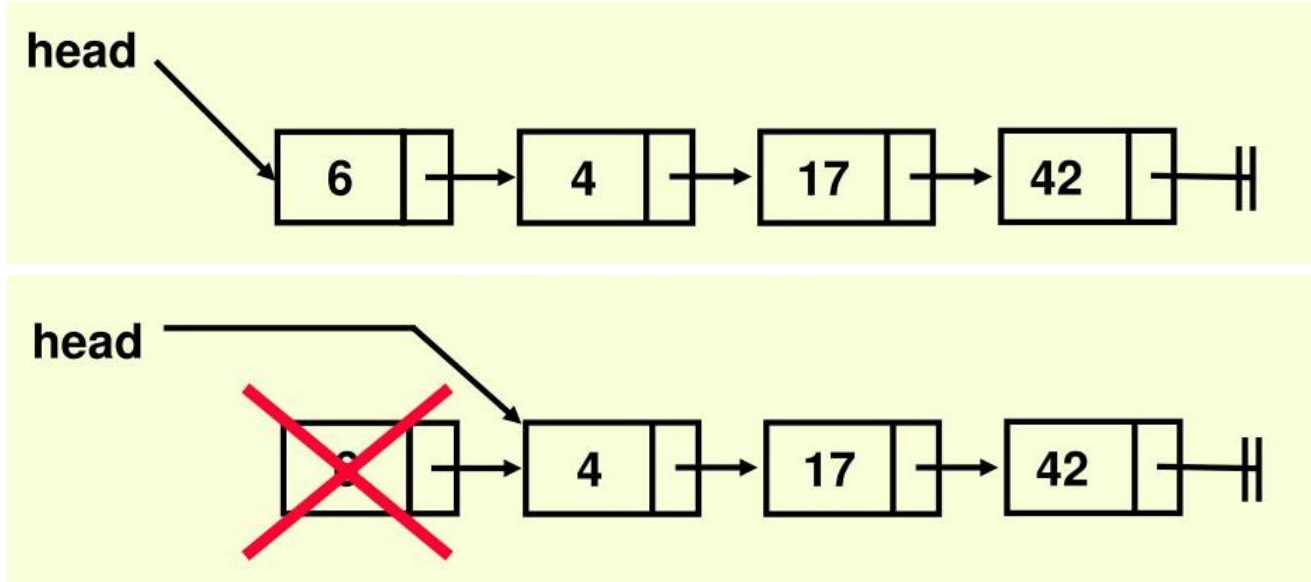
Pair Programming

Design Linked List (implement **addAtAnyIndex**)

Delete a node from the linked list

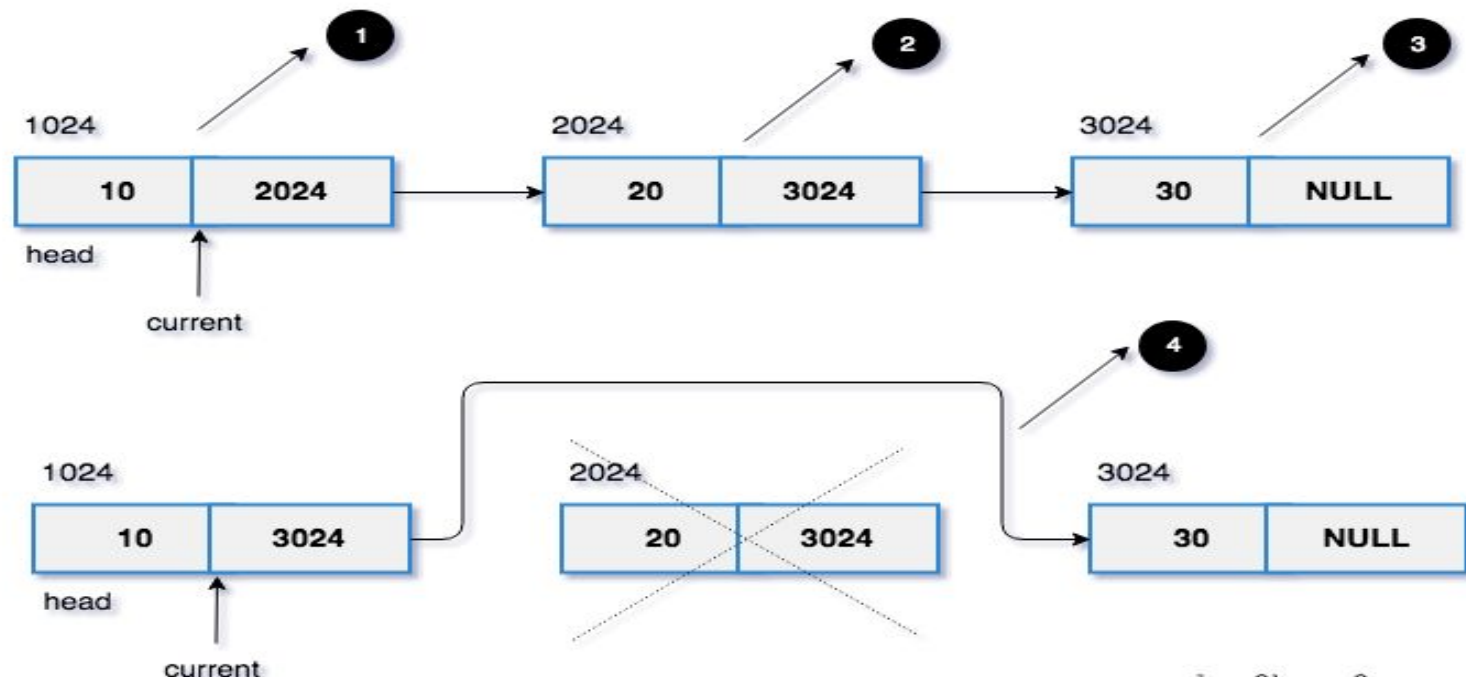
Delete a node at the beginning

- make the **second node as head**
- **Discard the memory allocated** for the first node.



Delete a node at any position

- **Find a match** the node to be deleted
- Get the **previous node**
- Make the **previous node next** point to the **next of the deleted node**



Can we avoid using two approaches when we are deleting nodes? How?

Yes, we can use Dummy Node.

Pair Programming

[Design Linked List](#) (implement **deleteAtIndex**)

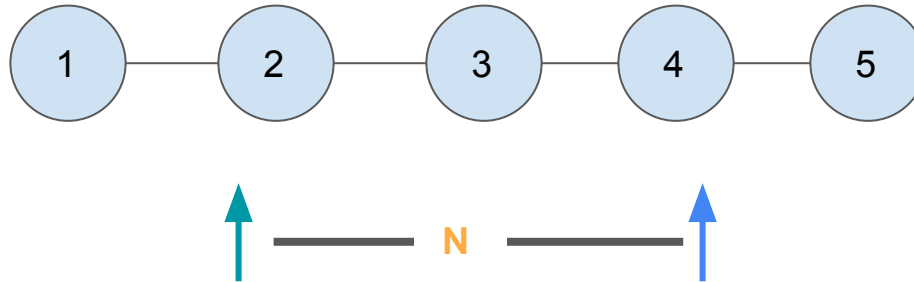
Two Pointer Technique in Linked List

Problem I

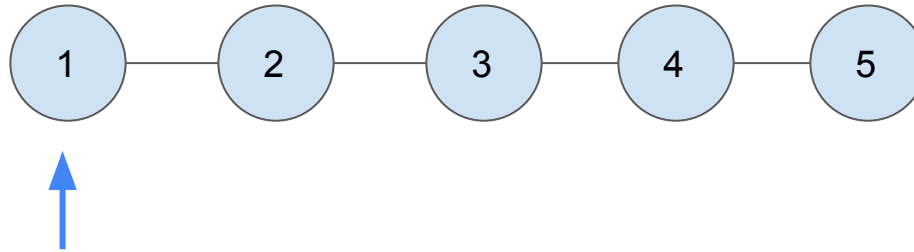
Return the nth last element of a singly linked list.

Approach I

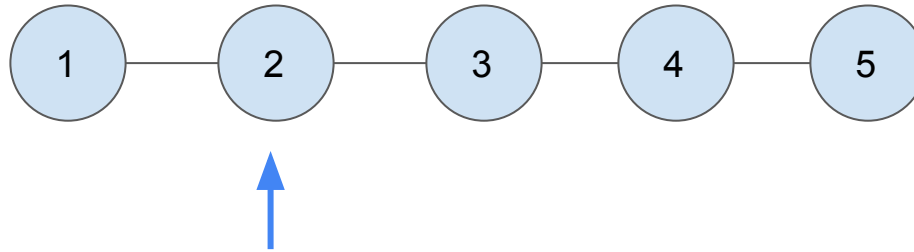
- iterate with two pointers
- one seeking the tail and the other lagging by the nth amount.



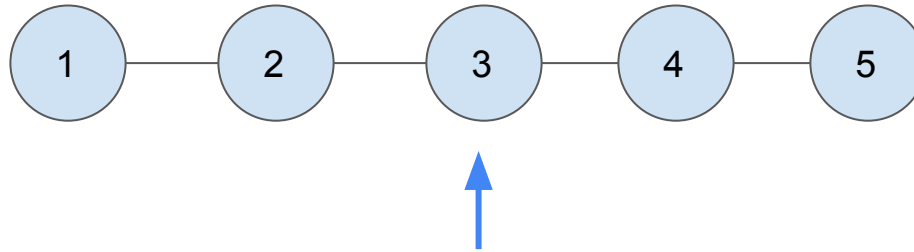
Simulation



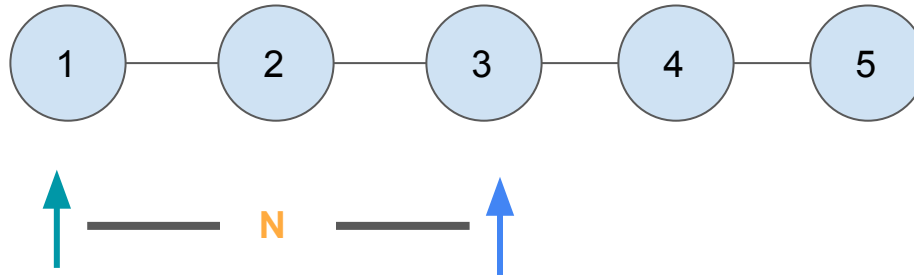
Simulation



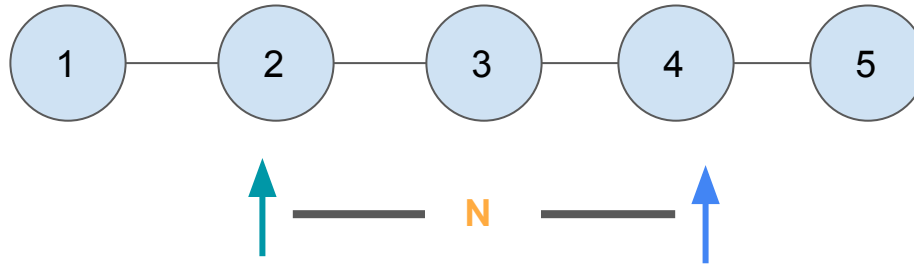
Simulation



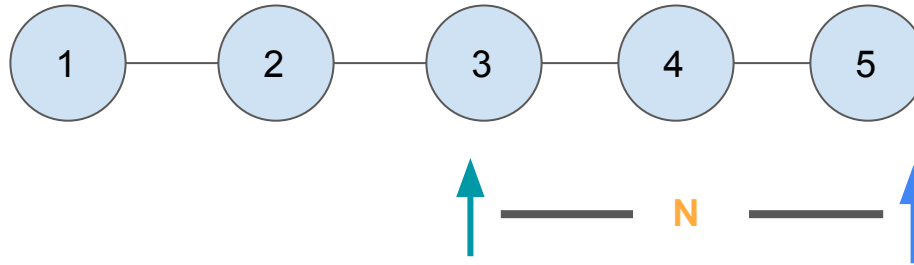
Simulation



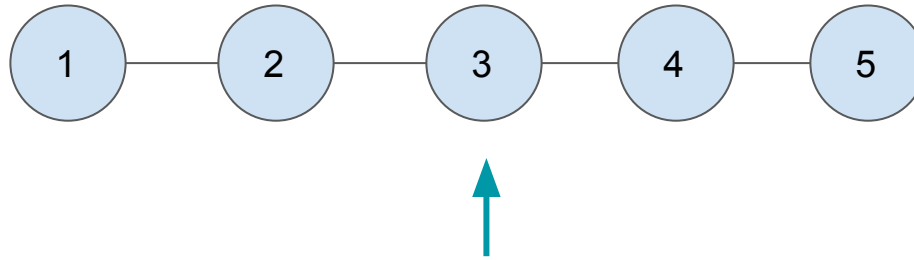
Simulation



Simulation



Simulation



Nth element from right

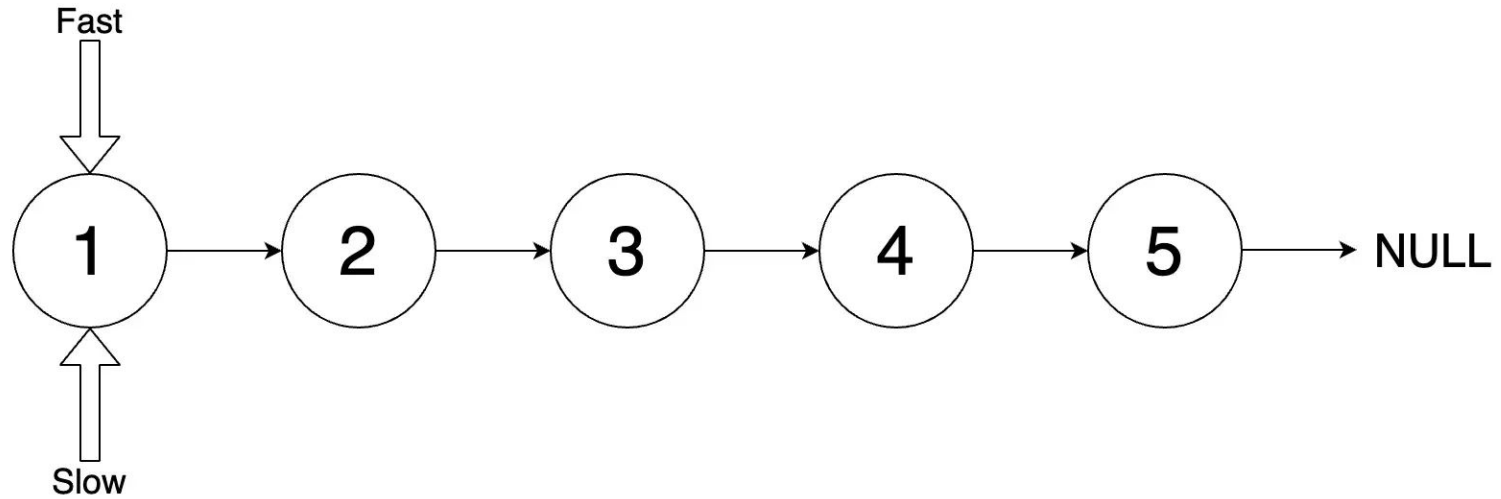
Problem II

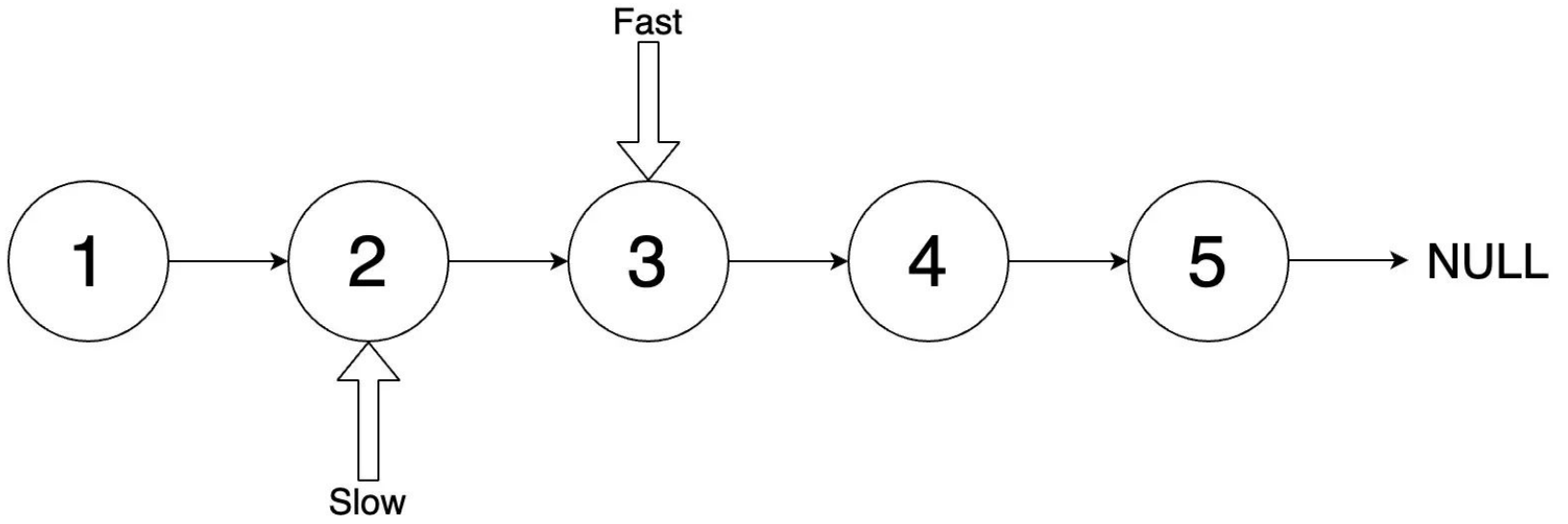
Return the middle element of a linked List.

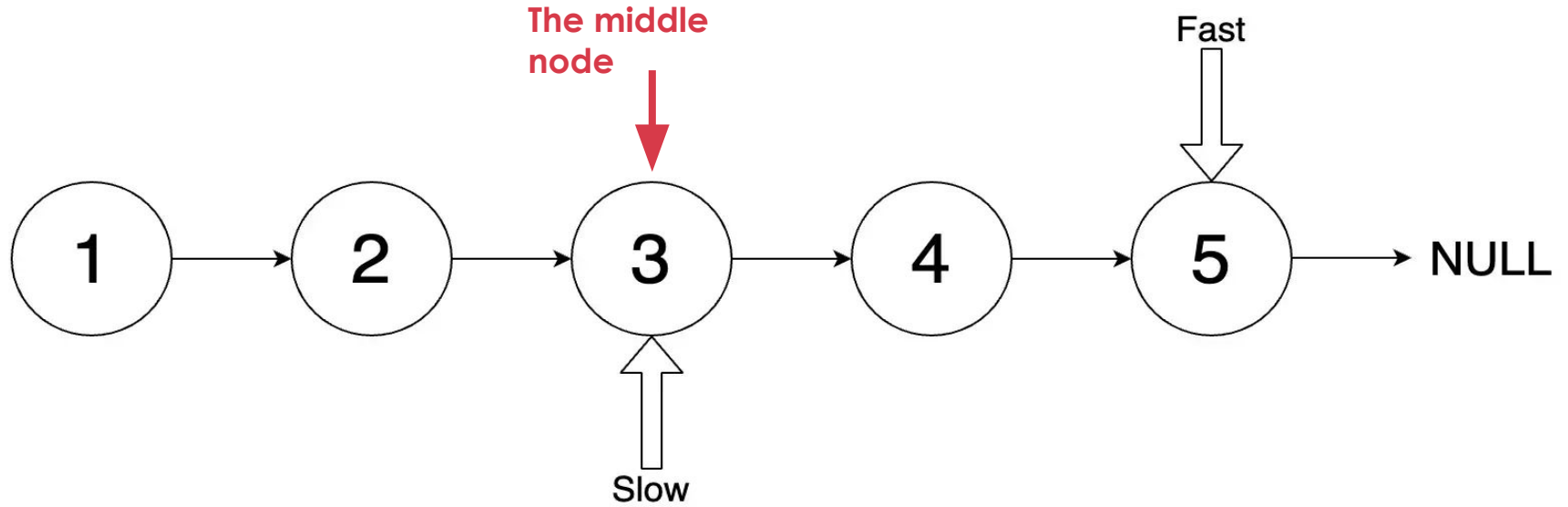
Approach : Fast and slow Pointers

- Iterate with **two pointers**: Fast and slow
- Slow pointer goes **one step** at a time
- Fast goes **two steps** at a time
- When the fast pointer moves to the very end of the Linked List, the slow pointer is going to point to the middle element of the linked list.

Simulation







Pair Programming

Middle element of a linked list

Floyd's Cycle Finding Algorithm

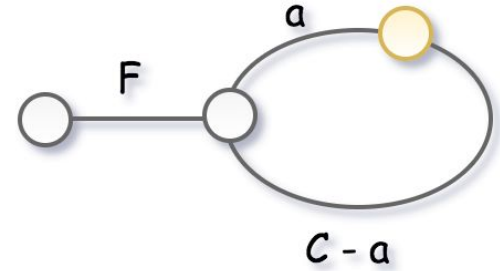
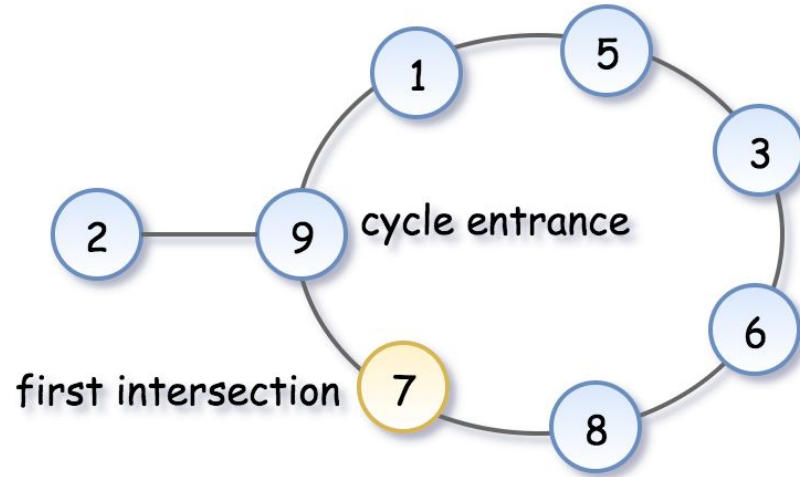
Problem

Given a linked list, return the node where the cycle begins?

Floyd's algorithm consists of two phases and uses two pointers, usually called **tortoise** and **rabbit**.

Phase I

- **Rabbit = cur.next.next** goes twice as fast as **tortoise = cur.next**
- Since tortoise is moving slower, the rabbit catches up to the tortoise at some **intersection** point
- Note that the **intersection point is not the cycle entrance** in the general case.



Phase II

To compute the intersection point, let's note that **the rabbit has traversed twice as many nodes as the tortoise**, i.e.

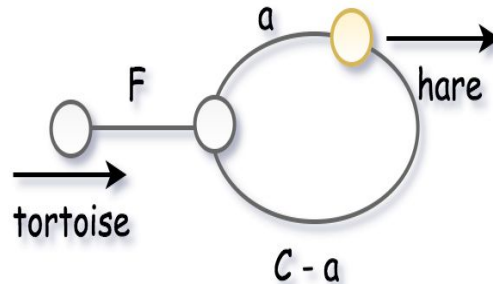
$2 \cdot d \cdot (\text{tortoise}) = d \cdot (\text{rabbit})$, implying:

$2 \cdot (F + a) = F + n \cdot C + a$, where n is some integer.

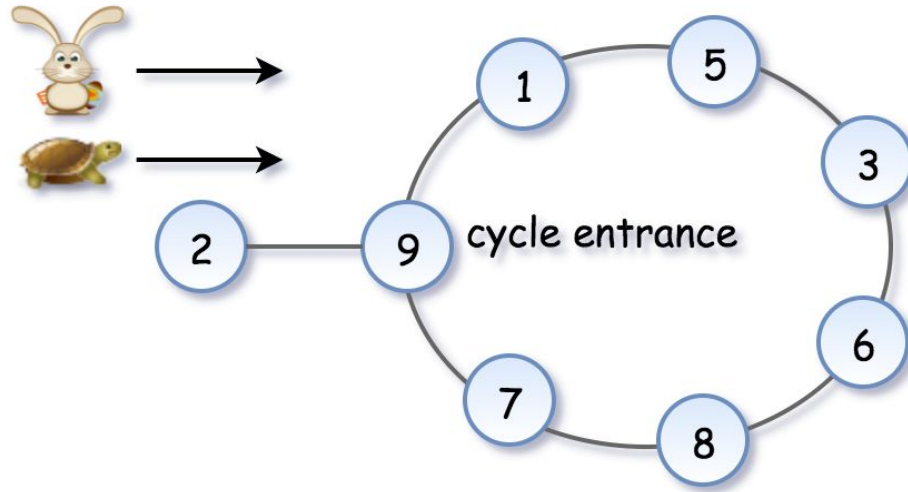
Hence the coordinate of the intersection point is

$$F + a = n \cdot C$$

- we give the tortoise a second chance by slowing down the rabbit,
 - **tortoise = tortoise.next**
 - **rabbit = rabbit.next**
- The tortoise is back at the starting position, and the rabbit start

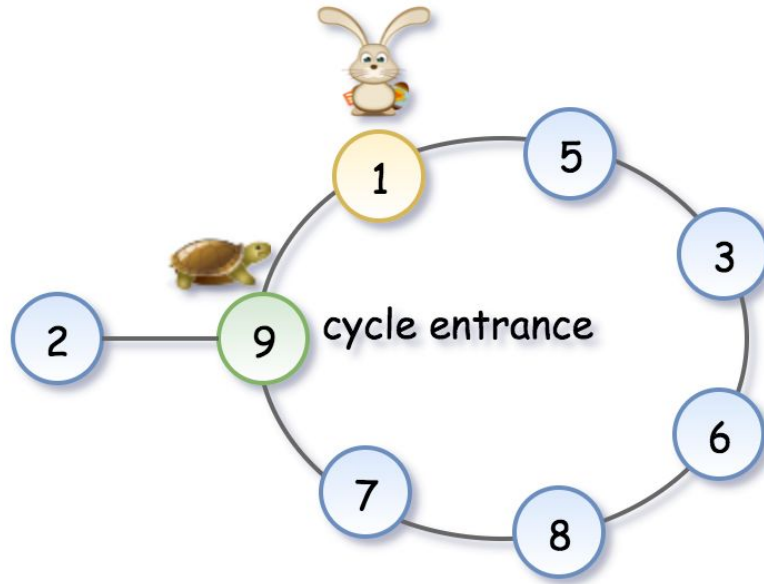


Phase 1



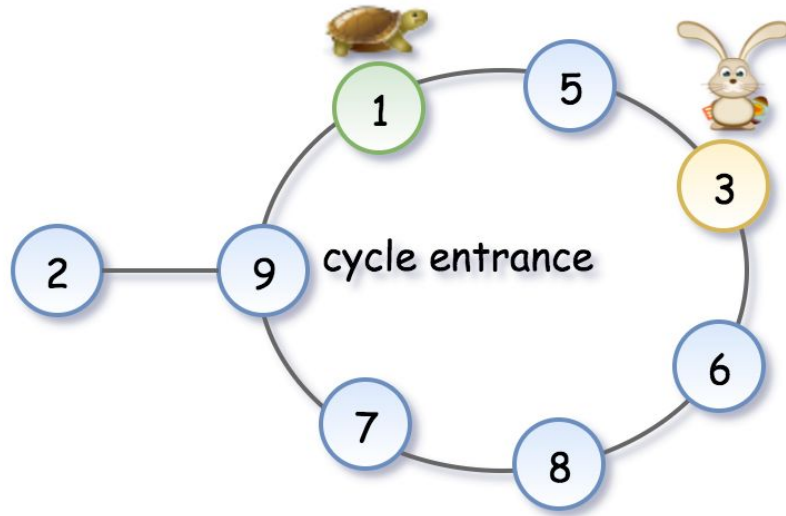
Phase 1

Phase 1



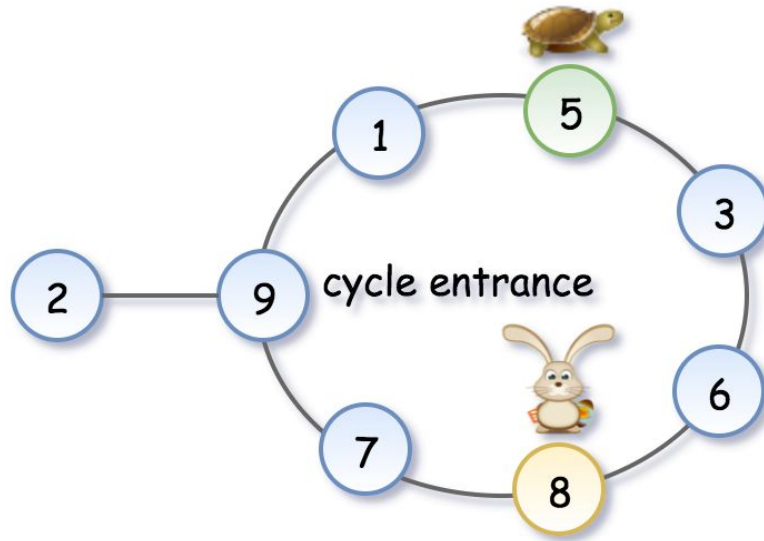
Phase 1

Phase 1



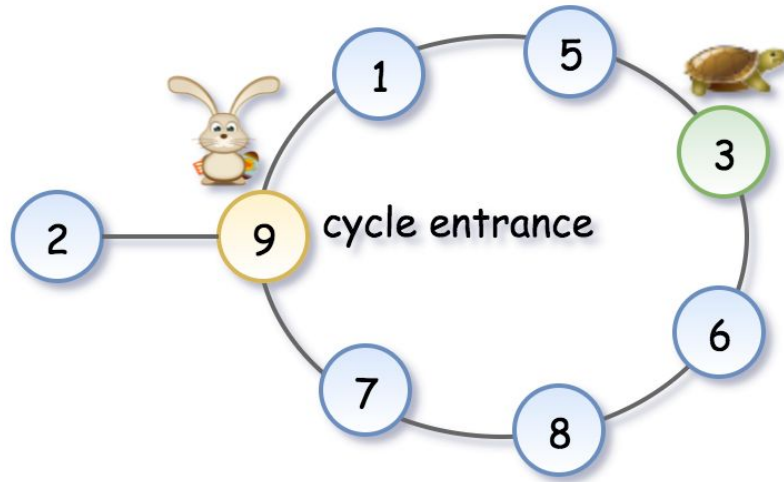
Phase 1

Phase 1

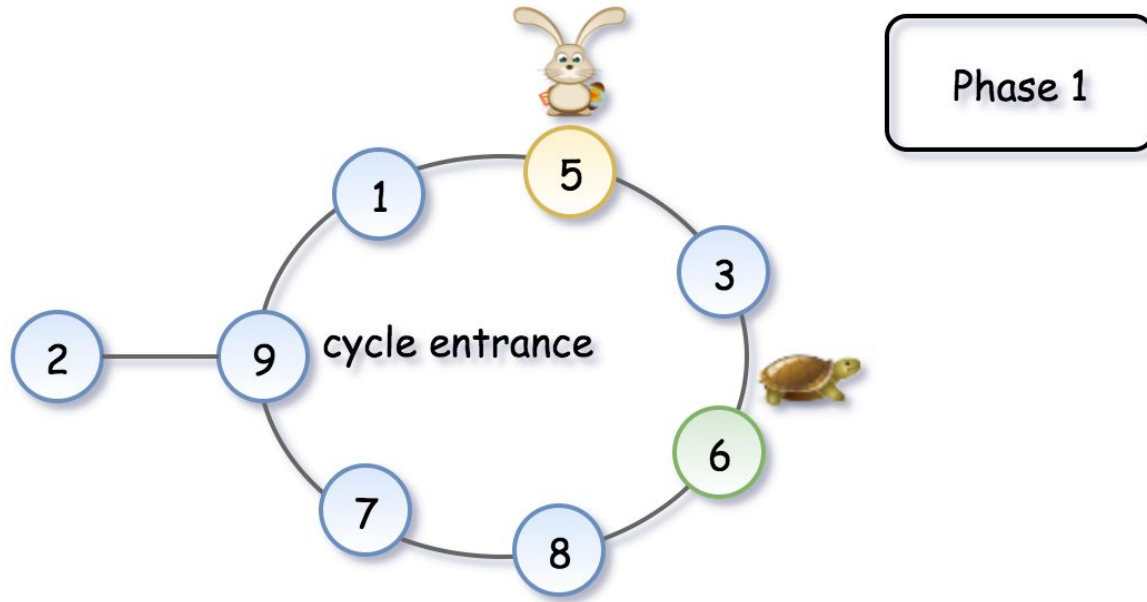


Phase 1

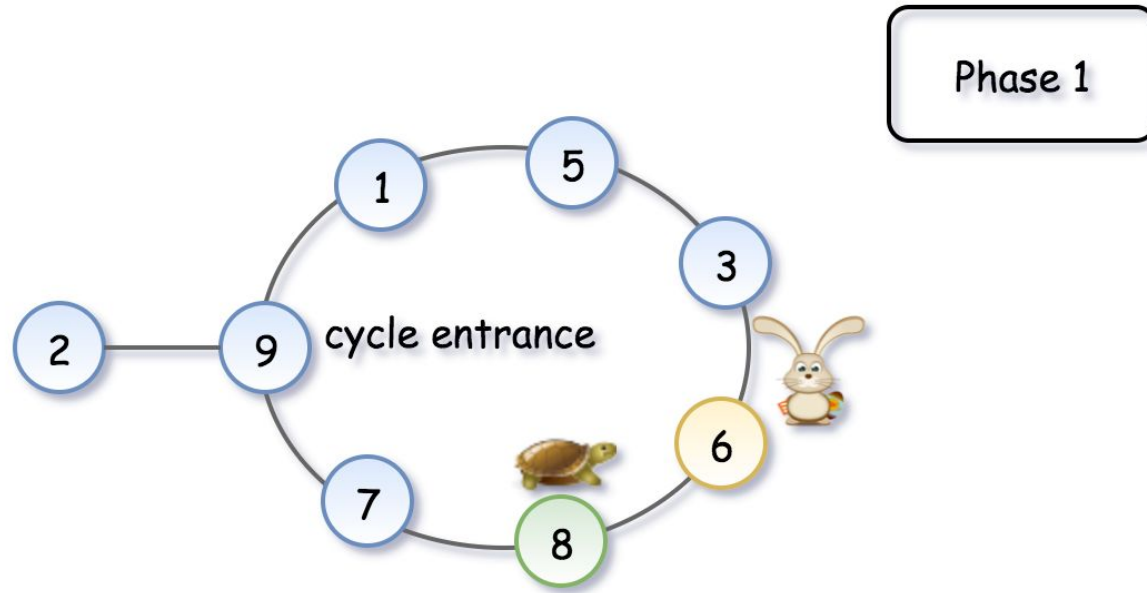
Phase 1



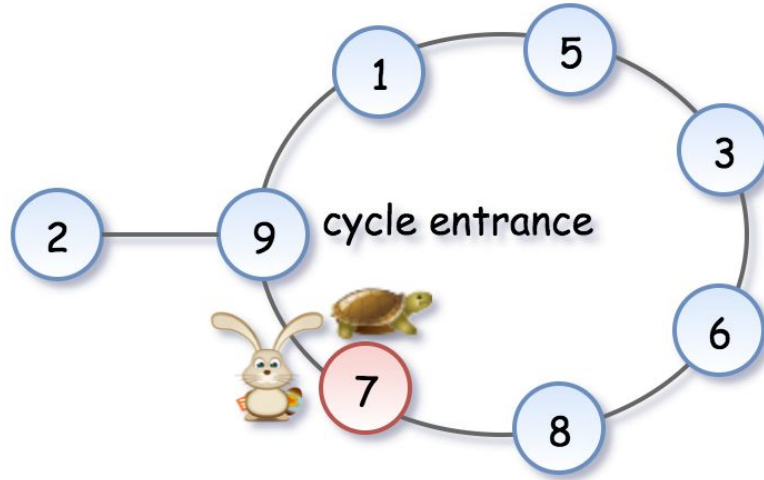
Phase 1



Phase 1

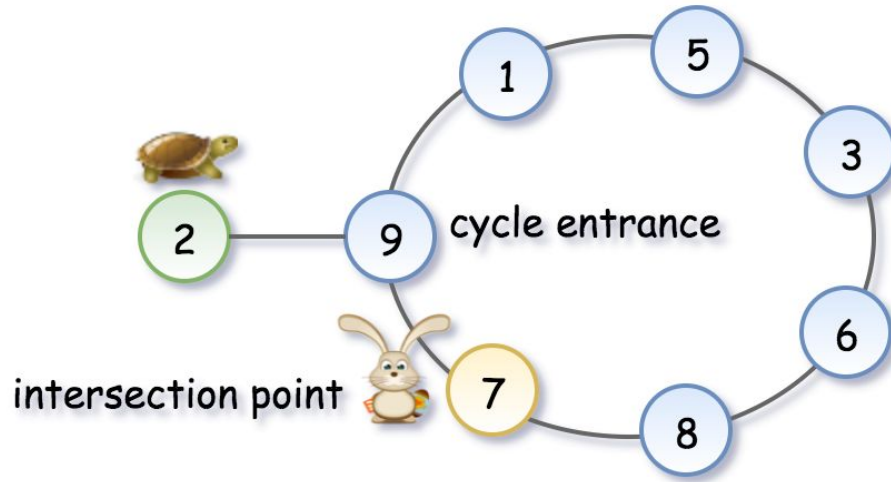


Phase 1 is over!



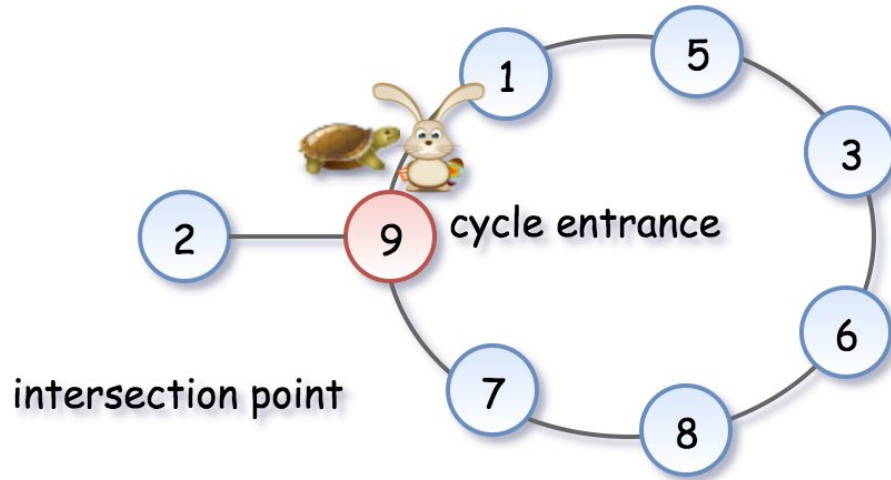
Phase 1
is over!

Phase 2



Phase 2

Phase 2 is over!



Phase 2
is over!

Rabbit meets tortoise -->
return 9

Mathematical Proof

Let x = part before the cycle

Let y = part until the meeting point

Let z = part starting from the first meeting point

$$L = x + z$$

$$\text{Fast} = x + c_1L + y$$

$$\text{Slow} = x + c_2L + y$$

$$2\text{Slow} = \text{Fast}$$

$$2(x + c_2L + y) = x + c_1L + y$$

... after some rearrangements

$$X = (\text{some constant number of moves}) * L + y$$

Pair Programming

Linked List Cycle

Things to pay attention

Pitfalls

- **Null pointer**
 - `head.next` <> check existence of head
 - `head.next.next` <> check existence of head and `head.next`
- **Losing the reference to head of the linked list**
 - Put the head in a variable before any operation to return the head at the end

Null Pointer Example

```
def containsTarget(head, target):  
    while head.val != target:  
        head = head.next  
  
    return head.val == target
```

What if head
is null?

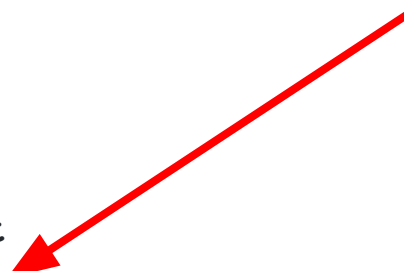
Null Pointer Example

```
def containsTarget(head, target):  
    while head and head.val != target:  
        head = head.next  
  
    return head.val == target
```

Null Pointer Example

```
def middleNode(self, head):  
    slow = head  
    fast = head  
  
    while fast:  
        slow = slow.next  
        fast = fast.next.next  
    return slow
```

What if
fast.next is
null?



Null Pointer Example

```
def middleNode(self, head):  
    slow = head  
    fast = head  
  
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
    return slow
```

losing the reference to head of the linked list Example

```
def deleteNodeAtGivenPosition(position, head) :  
    if head is None:  
        return  
  
    index = 0  
    previous = None  
    while head.next and index < position:  
        previous = head  
        head = head.next  
        index += 1  
    previous.next = head.next  
    return head
```

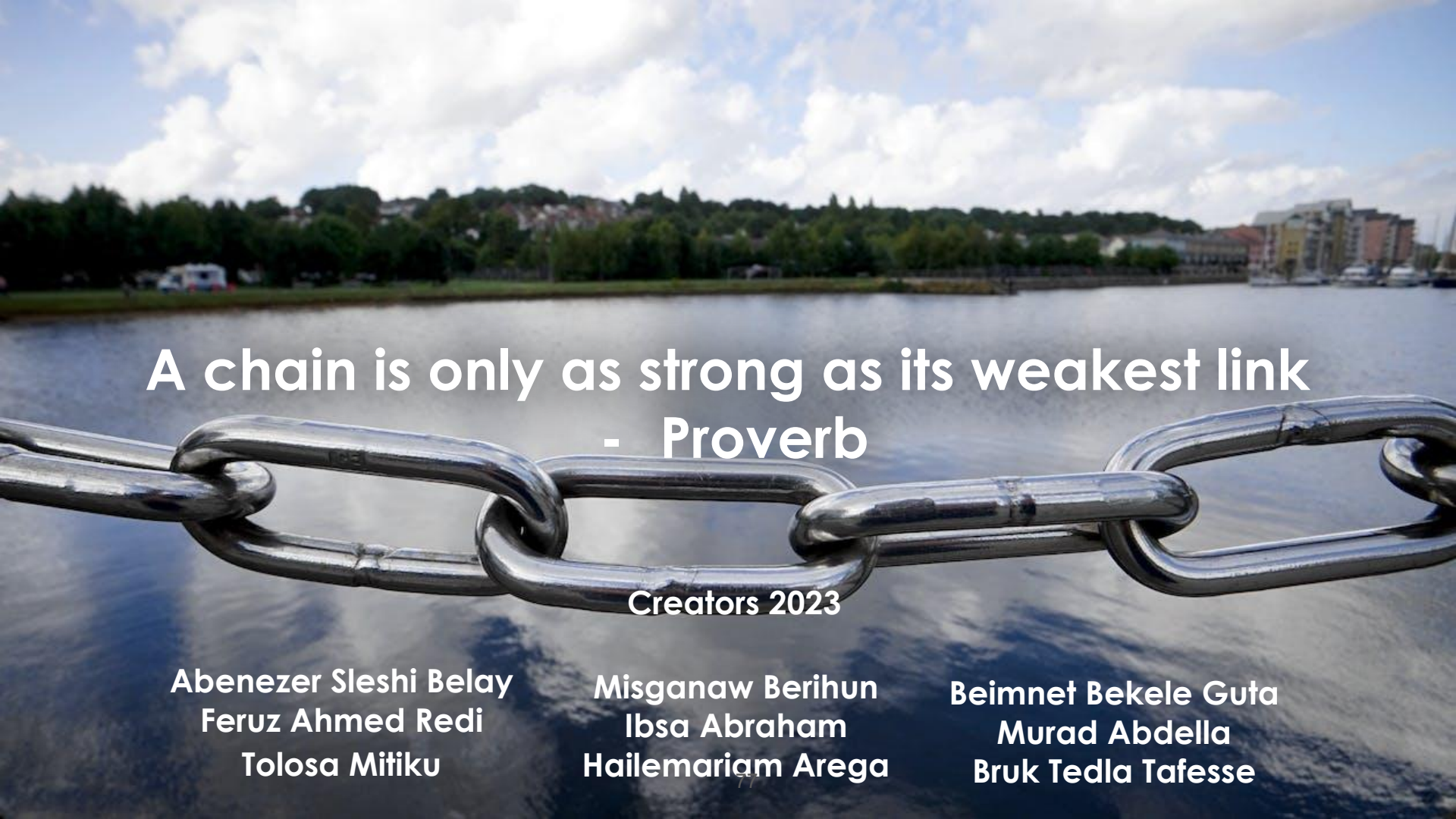
Is head the head of
the modified linked
list?

Practice Questions

- Design
 -
- Reverse Linked List
 - [Reverse Linked Lists](#)
 - [Reverse Linked List II](#)
 - [Palindrome Linked List](#)
- Reordering the nodes
 - [Partition List](#)
 - [Rotate List](#)
- Delete Nodes
 - [Delete Node in a Linked List](#)
 - [Remove Nth Node from end of list](#)
 - [Remove duplicates from sorted list](#)
- Sort linked lists
 - a. [Insertion Sort](#)
- Two Pointers
 - a. Parallel Pointers
 - i. [Add Two Numbers](#)
 - ii. [Merge Two Sorted Lists](#)
 - b. Fast and Slow Pointers
 - i. [Middle of linked list](#)
 - ii. [Linked List Cycle](#)
 - iii. [Linked List Cycle II](#)
- Miscellaneous
 - a. [Next Greater Node in a linked list](#)
 - b. [Odd Even Linked List](#)

Resources

- [Leetcode Explore Card](#): has excellent track path with good explanations
- Leetcode Solution ([Find the Duplicate Number](#)) : has good explanation about Floyd's cycle detection algorithm with good simulation
- Elements of Programming Interview book: has a very good Linked List Problems set



A chain is only as strong as its weakest link - Proverb

Creators 2023

Abenezer Sleshi Belay
Feruz Ahmed Redi
Tolosa Mitiku

Misganaw Berihun
Ibsa Abraham
Hailemariam Arega

Beimnet Bekele Guta
Murad Abdella
Bruk Tedla Tafesse